

Programmering med mikrokontroller

Aut2602 Innlevering 2

Jens Christian Valen Leynse

September 2024

1 Introduksjon

I denne oppgaven skal det leses av en analog verdi fra temperatursensoren på AVR128DB48-mikrokontrolleren og sende denne verdien til en datamaskin via USART. Målet med oppgaven er å bruke ADC (Analog til Digital Converter) for å fange opp temperaturmålinger og deretter overføre disse målingene i Celsius til en datamaskinterminal med bruk av UART-kommunikasjon.

Oppgaven inkluderer konfigurasjon av ADC og USART på mikrokontrolleren, samt implementering av funksjoner som muliggjør kontinuerlige målinger av temperatur som sendes over til terminalen hvert sekund. Videre, som extraoppgave, sammenlignes nøyaktigheten til den interne temperatursensoren med en ekstern sensor, TMP235.

2 Kode segmenter

Denne header-filen deklarerer funksjonene som brukes til å konfigurere ADC.

```
1 #ifndef ADC_H_
2 #define ADC_H_
3
4 #include <avr/io.h>
5 #include <stdbool.h>
6
7 void ADC_Init(void);
8 void ADC_Start(void);
9 uint16_t ADC_GetResult(void);
10 uint16_t Read_Temperature(void);
11 uint16_t Read_Extern_Temperature(void);
12 uint16_t Read_Intern_Temperature(void);
13
14 #endif /* ADC_H_ */
```

Funksjonen setter opp ADC-enheten på AVR-mikrokontrolleren med en referansespenning på 2.048V og 12-bit oppløsning. Her blir også en preskaler på 16 valgt, som sikrer en ADC-klokkefrekvens på 250kHz.

```
1 #include "adc.h"
2 #include <avr/io.h>
3
4 #define SCALING_FACTOR 4096
5
6 void ADC_Init(void) {
7     VREF.ADCOREF = VREF_REFSEL_2V048_gc |
8         VREF_ALWAYS_ON_bm;
9
10    // Set ADC clock, sample length, and
11    // initialization delay
12    ADC0.CTRLA = ADC_ENABLE_bm | ADC_FREERUN_bm |
13        ADC_RESSEL_12BIT_gc; // Enable ADC, freerunning
14    // makes startconversion only necessary in init
15    ADC0.CTRLC = ADC_PRESC_DIV16_gc; // Set prescaler
16    // for 250kHz
17    // ADC0.MUXPOS = MUXPORT; // Select temperature
18    // sensor as input
19    // ADC0.MUXPOS |= ADC_MUXPOS_TEMPSENSE_gc;
20    // ADC0.MUXPOS = ADC_MUXPOS_AIN5_gc;
21 }
```

Funksjonen starter ADC-konverteringen ved å sette kommando-flagget i ADC-enheten. Dette gjør at ADC begynner å lese signalet fra den valgte analoge inngangen.

```
1 // Start the conversion
2 void ADC_Start(void) {
3     ADC0.COMMAND = ADC_STCONV_bm;
4 }
```

Funksjonen sørger for å hente resultatet fra ADC-enheten etter at konverteringen er ferdig. Det sjekkes om ADC-flagget for ferdig konvertering er satt før verdien returneres.

```
1 uint16_t adc_get_result(){
2     while(!ADC0.INTFLAGS & ADC_RESRDY_bm)
3         ;
4     return ADC0.RES;
5 }
```

Funksjonen leser ADC-verdien fra den eksterne temperatursensoren og konverterer denne til temperatur i Celsius ved hjelp av en spenningskonverteringsformel.

```

1 // Read external temperature sensor from TMP235
2 uint16_t Read_Extern_Temperature(void) {
3     // Get ADC result
4     uint16_t adc_value = adc_get_result();
5
6     // Convert ADC value to temperature in Celsius
7     float voltage = (adc_value / 4096.0) * 2.048;
8     float temperature = (voltage - 0.5) * 100;
9
10    return (uint16_t)temperature;
11 }

```

Funksjonen bruker kalibreringsverdier fra Signature Row for å beregne temperaturen i Celsius. Den interne sensorens verdi blir korrigert for å gi et nøyaktig temperaturopslag.

```

1 // Read internal temperature
2 uint16_t Read_Intern_Temperature(void) {
3     // Retrieve calibration values from Signature Row
4     uint16_t sigrow_offset = SIGROW.TEMPSENSE1;
5     uint16_t sigrow_slope = SIGROW.TEMPSENSE0;
6
7     // Get ADC result
8     uint16_t adc_value = adc_get_result();
9
10    uint32_t temp = sigrow_offset - adc_value; //
        Kelvin
11    temp *= sigrow_slope;
12    //temp += SCALING_FACTOR / 2;
13    //temp /= SCALING_FACTOR;
14    temp += 0x0800;
15    temp >>= 12;
16
17    uint16_t temperature_in_C = temp - 273;
18    return temperature_in_C;
19
20 }

```

Hovedprogrammet starter ADC og USART og setter opp knappens konfigurasjon. Det veksler mellom lesing fra intern og ekstern temperatursensor avhengig av knappens tilstand, og sender resultatet til en datamaskin via USART for logging. Multiplexeren for valg av ADC-inngang (MUXPOS) ble flyttet til hovedprogrammet for å gjøre sensoren valgbart med en enkel "toggle"-funksjon, slik at man kan bytte mellom intern og ekstern temperatursensor ved å trykke på knappen.

```

1 #define F_CPU 4000000UL
2
3 #include <avr/io.h>
4 #include <util/delay.h>
5 #include "usart.h"
6 #include "adc.h"
7
8 // #define TEMPSENSE_ANALOG_PORT 0x42
9 #define BUTTON_PIN 2 // Button connected to PB2
10
11 void setup_button(void) {
12     // Set PB2 as an input (for the button)
13     PORTB.DIRCLR = (1 << BUTTON_PIN);
14
15     // Enable pull-up resistor on PB2 (if button
16     // connects to ground when pressed)
17     PORTB.PIN2CTRL = PORT_PULLUPEN_bm;
18 }
19
20 uint8_t read_button_state(void) {
21     return (PORTB.IN & (1 << BUTTON_PIN)) ? 1 : 0; //
22     // Return 1 if button not pressed, 0 if pressed
23 }
24
25 int main(void) {
26     ADC_Init();
27     usart_usb_init();
28     setup_button();
29
30     ADC_Start();
31
32     uint8_t sensor_select = 0;
33
34     while (1) {
35         if (read_button_state() == 0) { // Button
36             // pressed
37             sensor_select ^= 1; // Toggle between 0
38             // and 1
39             _delay_ms(300);
40         }
41         uint16_t temperature;
42         if (sensor_select == 0) {
43             ADC0.MUXPOS = ADC_MUXPOS_TEMPSENSE_gc;
44             temperature = Read_Intern_Temperature();
45             // Prepare the string to send via USART

```

```

42         printf("Intern temperature: %d C\n",
43                temperature);
44     } else {
45         ADC0.MUXPOS = ADC_MUXPOS_AIN5_gc;
46         temperature = Read_Extern_Temperature();
47         // Prepare the string to send via USART
48         printf("Extern temperature: %d C\n",
49                temperature);
50     }
51     _delay_ms(1000);
52 }
53 return 0;
54 }

```